Lab Sheet-06
Data Structure I (BTCS209P)
B.Tech III Semester (Session 2026-2027)

Objective:

To understand and implement the fundamental Queue data structure using C++, and to analyze its working, algorithms, time complexity, space complexity, and practical applications.

Requirements:

C++ Compiler / IDE

VS Code, Code::Blocks, Dev-C++, or any suitable C++ environment

Basic knowledge of arrays, loops, functions, and conditional statements

Concepts Covered:

1. Queue and FIFO Principle
2. Queue Representation using Array
3. Enqueue Operation
4. Dequeue Operation
5. Peek Operation
6. Display Operation
7. Queue Overflow and Underflow
8. Circular Queue
9. Queue Applications

1. Queue

A Queue is a linear data structure in which insertion is performed at one end called Rear and deletion is performed at the other end called Front. Queue follows the FIFO (First In First Out) principle.

2. FIFO (First In First Out)

In a Queue, the element inserted first is removed first.

Example: Front → [10] [20] [30] [40] ← Rear

3. Queue Representation using Array

A Queue can be represented using an array with two variables, Front and Rear. Initially, Front = -1 and Rear = -1.

For a queue of size 5:

Index: 0 1 2 3 4

Queue: [10] [20] [30] [40] [50]

↑ ↑

Front Rear

4. Enqueue Operation

Enqueue is the operation used to insert a new element into the Queue at the Rear end.

Algorithm:

ENQUEUE(Q, value)

1. Check if Rear == MAX - 1.
2. If yes, display "Queue Overflow" and stop.
3. If Front == -1, set Front = 0.
4. Increment Rear by 1.
5. Set Q[Rear] = value.
6. Stop.

Time and Space Complexity:

Time: O(1), Space: O(n) for array storage.

Properties:

Enqueue inserts elements at the Rear end and follows FIFO ordering.

5. Dequeue Operation

Dequeue is the operation used to remove an element from the Queue from the Front end.

Algorithm:

DEQUEUE(Q)

1. Check if Front == -1 or Front > Rear.
2. If yes, display "Queue Underflow" and stop.
3. Store or display Q[Front].
4. Increment Front by 1.
5. If Front > Rear, reset Front = Rear = -1.
6. Stop.

Time and Space Complexity:

Time: O(1), Space: O(1) auxiliary space.

Properties:

Dequeue removes the element that has been present in the Queue for the longest time.

6. Peek Operation

Peek displays the element at the Front without removing it.

Algorithm:

PEEK(Q)

1. Check if the Queue is empty.
2. If yes, display "Queue is Empty".
3. Otherwise, display Q[Front].
4. Stop.

Time and Space Complexity:

Time: O(1), Space: O(1).

7. Display Operation

Display traverses the Queue from Front to Rear and displays all elements.

Algorithm:

DISPLAY(Q)

1. Check if the Queue is empty.
2. If yes, display "Queue is Empty".
3. Otherwise, traverse from Front to Rear.
4. Display each element.
5. Stop.

Time and Space Complexity:

Time: O(n), Space: O(1) auxiliary space.

8. Queue Overflow and Underflow

Queue Overflow occurs when an insertion is attempted in a full linear array Queue. Queue Underflow occurs when a deletion is attempted from an empty Queue.

9. Complete C++ Array Implementation

```
#include <iostream>
using namespace std;

#define MAX 5

class Queue {
   int q[MAX];
   int front, rear;

public:
```

```
Queue() {
    front = -1;
    rear = -1;
}

void enqueue(int value) {
    if (rear == MAX - 1) {
        cout << "Queue Overflow!" << endl;
        return;
    }
    if (front == -1)
        front = 0;
    q[++rear] = value;
    cout << value << " inserted." << endl;
}

void dequeue() {
    if (front == -1 || front > rear) {
        cout << "Queue Underflow!" << endl;
        return;
    }
    cout << q[front] << " deleted." << endl;
    front++;
    if (front > rear)
        front = rear = -1;
}

void peek() {
    if (front == -1)
        cout << "Queue is Empty!" << endl;
    else
        cout << "Front element: " << q[front] << endl;
}

void display() {
    if (front == -1) {
        cout << "Queue is Empty!" << endl;
        return;
    }
    cout << "Queue: ";
    for (int i = front; i <= rear; i++)
        cout << q[i] << " ";
    cout << endl;
}
};

int main() {
    Queue q;
```

```
    int choice, value;

    do {
        cout << "\n--- QUEUE MENU ---\n";
        cout << "1. Enqueue\n2. Dequeue\n3. Peek\n";
        cout << "4. Display\n5. Exit\n";
        cout << "Enter choice: ";
        cin >> choice;

        switch (choice) {
        case 1:
            cout << "Enter value: ";
            cin >> value;
            q.enqueue(value);
            break;
        case 2:
            q.dequeue();
            break;
        case 3:
            q.peek();
            break;
        case 4:
            q.display();
            break;
        case 5:
            cout << "Program terminated." << endl;
            break;
        default:
            cout << "Invalid choice!" << endl;
        }
    } while (choice != 5);

    return 0;
}
```

10. Time and Space Complexity

Enqueue: O(1) | Dequeue: O(1) | Peek: O(1) | Display: O(n) | Space: O(n)

11. Applications of Queue

1. CPU Scheduling
2. Printer Queue
3. Process Scheduling
4. Breadth First Search (BFS)
5. Call Center Systems
6. Network Data Packets
7. Keyboard / I/O Buffer
8. Task Scheduling

9. Customer Service Systems
10. Operating System Scheduling

12. Circular Queue

A Circular Queue connects the last position of the array back to the first position. It efficiently reuses the empty positions created after dequeue operations.

Circular Queue is useful when a fixed-size array Queue needs efficient utilization of available space.

Lab Exercises

1. Write a C++ program to implement a Queue using an array. Perform Enqueue, Dequeue, Peek and Display operations.

2. Write a C++ program to implement Enqueue operation and demonstrate insertion at the Rear.

3. Write a C++ program to implement Dequeue operation and demonstrate deletion from the Front.

4. Write a C++ program to implement Peek operation and display the Front element without deleting it.

5. Write a C++ program to display all elements of a Queue from Front to Rear.

6. Write a C++ program to demonstrate Queue Overflow using an array.

7. Write a C++ program to demonstrate Queue Underflow.

8. Write a menu-driven C++ program implementing all basic Queue operations.

9. Write a C++ program to count the number of elements currently present in a Queue.

10. Write a C++ program to search for a given element in a Queue.

11. Write a C++ program to find the maximum and minimum elements in a Queue.

12. Write a C++ program to reverse the elements of a Queue.

13. Write a C++ program to implement a Circular Queue using an array.

14. Write a C++ program to implement Queue using a Linked List.

15. Write a C++ program to compare Linear Queue and Circular Queue in terms of space utilization and operations.

Instructions & Guidelines

- Write a separate C++ program for each assigned exercise.
- Use functions wherever appropriate to make the programs modular and easy to understand.
- Do not use built-in Queue/STL functions for implementing the basic Queue operations in the assigned array implementation.
- Test each program with different inputs, including empty Queue, single element, and full Queue cases.
- Display the input and output clearly.
- Add comments to important parts of the program.
- Mention the time and space complexity of each implemented operation.
- For comparison-based exercises, use the same input conditions for a fair comparison.

Submission

Submit the C++ source code files for the assigned tasks along with a brief report containing the objective, algorithm, program, sample input/output, result, and time and space complexity analysis of the implemented Queue operations.